

```

class PuzzleState:
    def __init__(self, board, parent=None, move=None, depth=0):
        self.board = board          # 1D list of 9 elements representing 3x3 grid
        self.parent = parent         # Link to parent node to track the solution path
        self.move = move            # Direction taken to reach this state
        self.depth = depth          # Current depth level of the state

    def __eq__(self, other):
        return self.board == other.board

    def __hash__(self):
        return hash(tuple(self.board))

def get_neighbors(state):
    """Generates valid neighbor configurations by moving the empty tile (0)."""
    neighbors = []
    zero_idx = state.board.index(0)
    row, col = divmod(zero_idx, 3)

    # Move actions defined as (row_change, col_change, action_name)
    moves = [
        (-1, 0, 'Up'),
        (1, 0, 'Down'),
        (0, -1, 'Left'),
        (0, 1, 'Right')
    ]

    for dr, dc, move_name in moves:
        new_row, new_col = row + dr, col + dc

        if 0 <= new_row < 3 and 0 <= new_col < 3:
            new_zero_idx = new_row * 3 + new_col
            # Copy the board layout
            new_board = list(state.board)
            # Swap empty tile (0) with the target tile
            new_board[zero_idx], new_board[new_zero_idx] = new_board[new_zero_idx], new_board[zero_idx]

            neighbors.append(PuzzleState(new_board, state, move_name, state.depth + 1))

    return neighbors

def solve_8_puzzle_dfs(initial_board, goal_board, max_depth=30):
    """Solves the 8-puzzle using Depth-First Search up to a maximum depth."""
    start_state = PuzzleState(initial_board)

    # Using a Python list as a LIFO stack for DFS
    stack = [start_state]
    visited = set()

    while stack:
        current_state = stack.pop()

        if current_state.board == goal_board:
            return current_state # Found the goal state

        if current_state in visited:
            continue

        visited.add(current_state)

        # Enforce depth ceiling to prevent infinite branch exploration
        if current_state.depth < max_depth:
            # We explore neighbors; reversing keeps standard order execution
            for neighbor in reversed(get_neighbors(current_state)):
                if neighbor not in visited:
                    stack.append(neighbor)

    return None # Solution not found within limits

def print_solution(solution_state):
    """Backtracks from the goal state to print the execution steps."""
    path = []
    current = solution_state
    while current:
        path.append(current)
        current = current.parent
    path.reverse()

    print(f"Total Moves to Goal: {len(path) - 1}\n")
    for i, state in enumerate(path):
        if state.move:

```

```

    print(f"Step {i}: Move Blank '{state.move}''")
else:
    print("Initial Configuration:")

    # Format and display the 3x3 layout
    for j in range(0, 9, 3):
        print(state.board[j:j+3])
    print("-" * 15)

# --- Execution ---
if __name__ == "__main__":
    # 0 represents the empty block
    # Simple, highly structured initial state to ensure prompt DFS execution
    initial = [2, 8, 3,
               1, 6, 4,
               7, 0, 5]

    goal = [1, 2, 3,
            8, 0, 4,
            7, 6, 5]

    solution = solve_8_puzzle_dfs(initial, goal, max_depth=20)

    if solution:
        print_solution(solution)
    else:
        print("No solution found within the specified depth limit.")

```

Total Moves to Goal: 5

Initial Configuration:

```

[2, 8, 3]
[1, 6, 4]
[7, 0, 5]

```

Step 1: Move Blank 'Up'

```

[2, 8, 3]
[1, 0, 4]
[7, 6, 5]

```

Step 2: Move Blank 'Up'

```

[2, 0, 3]
[1, 8, 4]
[7, 6, 5]

```

Step 3: Move Blank 'Left'

```

[0, 2, 3]
[1, 8, 4]
[7, 6, 5]

```

Step 4: Move Blank 'Down'

```

[1, 2, 3]
[0, 8, 4]
[7, 6, 5]

```

Step 5: Move Blank 'Right'

```

[1, 2, 3]
[8, 0, 4]
[7, 6, 5]

```
